

tf.CriticalSection Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/CriticalSection

Critical section.

Function Signatures

```
tf.CriticalSection( name=None, shared_name=None,
critical_section_def=None, import_scope=None )
v = resource_variable_ops.ResourceVariable(0.0, name="v") def
count(): value = v.read_value() with
tf.control_dependencies([value]): with
tf.control_dependencies([v.assign_add(1)]): return
tf.identity(value)
```

Code Examples

```
tf.CriticalSection(
name=None, shared_name=None, critical_section_def=None,
import_scope=None
)
```

```
tf.CriticalSection(  
name=None, shared_name=None, critical_section_def=None,  
import_scope=None  
)
```

```
v = resource_variable_ops.ResourceVariable(0.0, name="v")  
def count():  
    value = v.read_value()  
    with tf.control_dependencies([value]):  
        with tf.control_dependencies([v.assign_add(1)]):  
            return tf.identity(value)
```

```
v = resource_variable_ops.ResourceVariable(0.0, name="v")  
def count():  
    value = v.read_value()  
    with tf.control_dependencies([value]):  
        with tf.control_dependencies([v.assign_add(1)]):  
            return tf.identity(value)
```

```
cs = CriticalSection()  
f1 = cs.execute(count)  
f2 = cs.execute(count)  
output = f1 + f2  
session.run(output)
```

```
cs = CriticalSection()  
f1 = cs.execute(count)  
f2 = cs.execute(count)  
output = f1 + f2  
session.run(output)
```

```

v = tf.compat.v1.get_variable("v", initializer=0.0,
use_resource=True)
def accumulate(up):
    x = v.read_value()
    with tf.control_dependencies([x]):
        with tf.control_dependencies([v.assign_add(up)]):
            return tf.identity(x)
    ex1 = CriticalSection().execute(
        accumulate, 1.0, exclusive_resource_access=False)
    ex2 = CriticalSection().execute(
        accumulate, 1.0, exclusive_resource_access=False)
    bad_sum = ex1 + ex2
    sess.run(v.initializer)
    sess.run(bad_sum)  # May return 0.0

```

```

v = tf.compat.v1.get_variable("v", initializer=0.0,
use_resource=True)
def accumulate(up):
    x = v.read_value()
    with tf.control_dependencies([x]):
        with tf.control_dependencies([v.assign_add(up)]):
            return tf.identity(x)
    ex1 = CriticalSection().execute(
        accumulate, 1.0, exclusive_resource_access=False)
    ex2 = CriticalSection().execute(
        accumulate, 1.0, exclusive_resource_access=False)
    bad_sum = ex1 + ex2
    sess.run(v.initializer)
    sess.run(bad_sum)  # May return 0.0

```

Documentation

Critical section.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.CriticalSection`

A `CriticalSection` object is a resource in the graph which executes subgraphs in serial order. A common example of a subgraph one may wish to run exclusively is the one given by the following function:

Here, a snapshot of `v` is captured in `value`; and then `v` is updated.
The snapshot value is returned.

If multiple workers or threads all execute `count` in parallel, there is no guarantee that access to the variable `v` is atomic at any point within any thread's calculation of `count`. In fact, even implementing an atomic counter that guarantees that the user will see each value `0, 1, ...,` is currently impossible.

The solution is to ensure any access to the underlying resource `v` is only processed through a critical section:

The functions `f1` and `f2` will be executed serially, and updates to `v` will be atomic.

All resource objects, including the critical section and any captured variables of functions executed on that critical section, will be colocated to the same device (host and `cpu/gpu`).

When using multiple critical sections on the same resources, there is no guarantee of exclusive access to those resources. This behavior is disallowed by default (but see the kwarg `exclusive_resource_access`).

For example, running the same function in two separate critical sections will not ensure serial execution:

Attributes

Methods

execute

[View source](#)

Execute function `fn()` inside the critical section.

`fn` should not accept any arguments. To add extra arguments to when calling `fn` in the critical section, create a lambda: